
Smart MCQ Solver Challenge

Deep Learning & Generative AI Project Report

Sanish Kumar (23f3001252) | IIT Madras BS Degree | Deep Learning & Generative AI

1. Abstract

This project presents an intelligent Multiple-Choice Question (MCQ) solving system that predicts the top-3 most probable answers using Natural Language Processing (NLP), Machine Learning, Deep Learning, and Retrieval-Augmented Generation (RAG). Development began with traditional machine learning approaches and progressed incrementally toward advanced Transformer-based architectures, with multiple experiments run along the way to compare model performance, optimize inference speed, and understand deployment trade-offs. The final solution combines a lightweight TF-IDF + Logistic Regression prediction model with Sentence-Transformer and FAISS retrieval, balancing prediction quality, explainability, and deployment efficiency.

2. Problem Statement

The objective is to automatically predict the correct answer for a Multiple Choice Question containing five answer options. Rather than predicting a single answer, the system generates Top-3 ranked predictions, evaluated using the Mean Average Precision at 3 (MAP@3) metric. In addition to prediction, the project retrieves similar questions from the training dataset via semantic search, so users can understand why a given prediction was made.

3. Dataset Description

The dataset consists of multiple-choice questions where each sample contains a question prompt, five answer choices (A - E), and a ground-truth answer. It was split into training and validation sets for model development and evaluation, with EDA performed beforehand to understand dataset characteristics, class distribution, and text properties.

The dataset consists of Multiple Choice Questions.

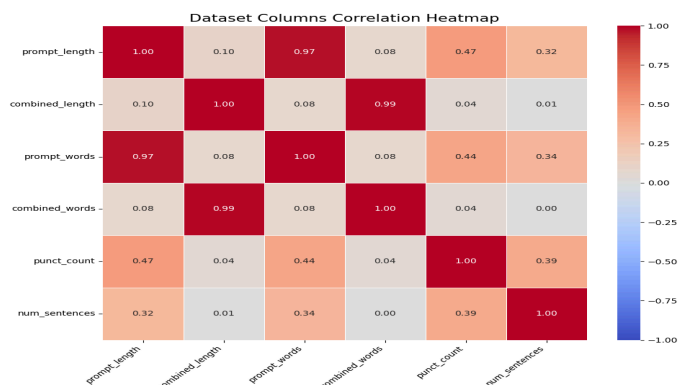
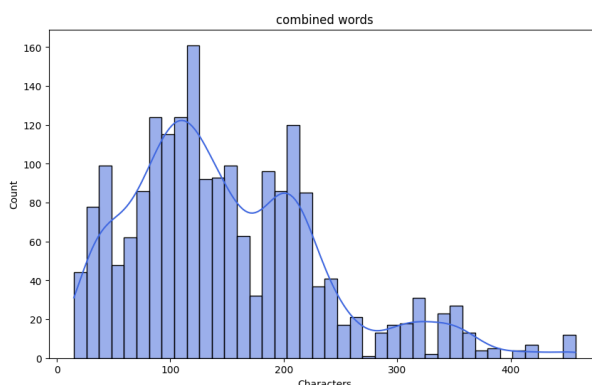
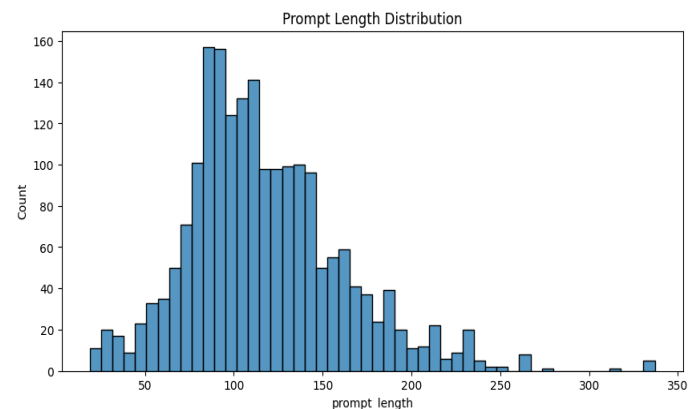
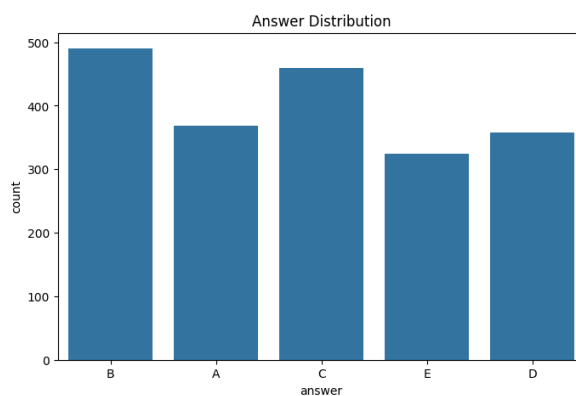
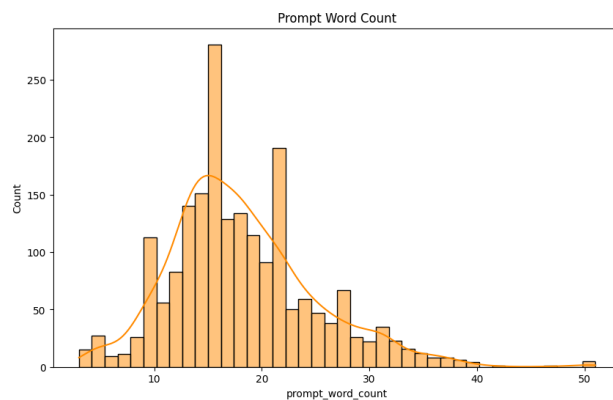
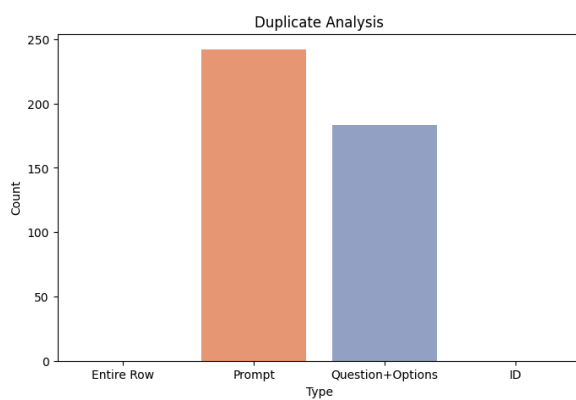
Each sample contains -

- Question
- Option A
- Option B
- Option C
- Option D
- Option E
- Correct Answer

4. Exploratory Data Analysis

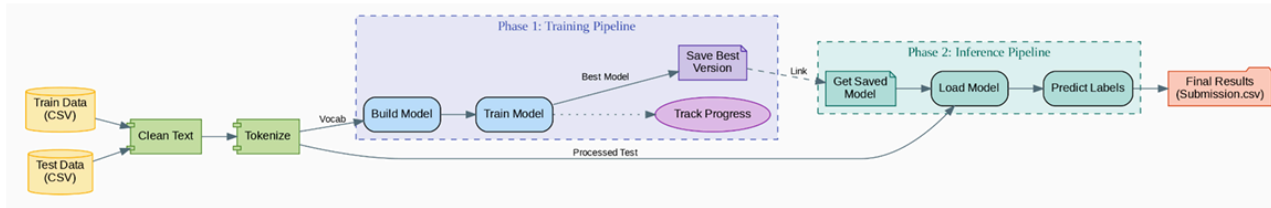
Exploratory Data Analysis was performed before building any predictive model, in order to surface data-quality issues and inform preprocessing choices. Statistical features such as question and option length were also generated to understand data variability. The following analyses were conducted:

- Missing-value analysis
- Class distribution
- Duplicate-question analysis
- Prompt / option length analysis
- Feature correlation
- Word-frequency visualization



5. Methodology

Overall Architecture



Rather than training one model directly, the project followed an incremental pipeline, comparing each model family before moving to the next stage.

6. Feature Engineering

Different preprocessing techniques were designed for each model family, so every model received input in the most suitable format. For traditional machine learning models, the question and all answer options were concatenated into a single document and transformed using TF-IDF vectorization. For Transformer models, the question and answer choices were separated using special tokens to preserve contextual information. For semantic retrieval, the question and options were converted into natural-language formatted text before generating Sentence-Transformer embeddings.

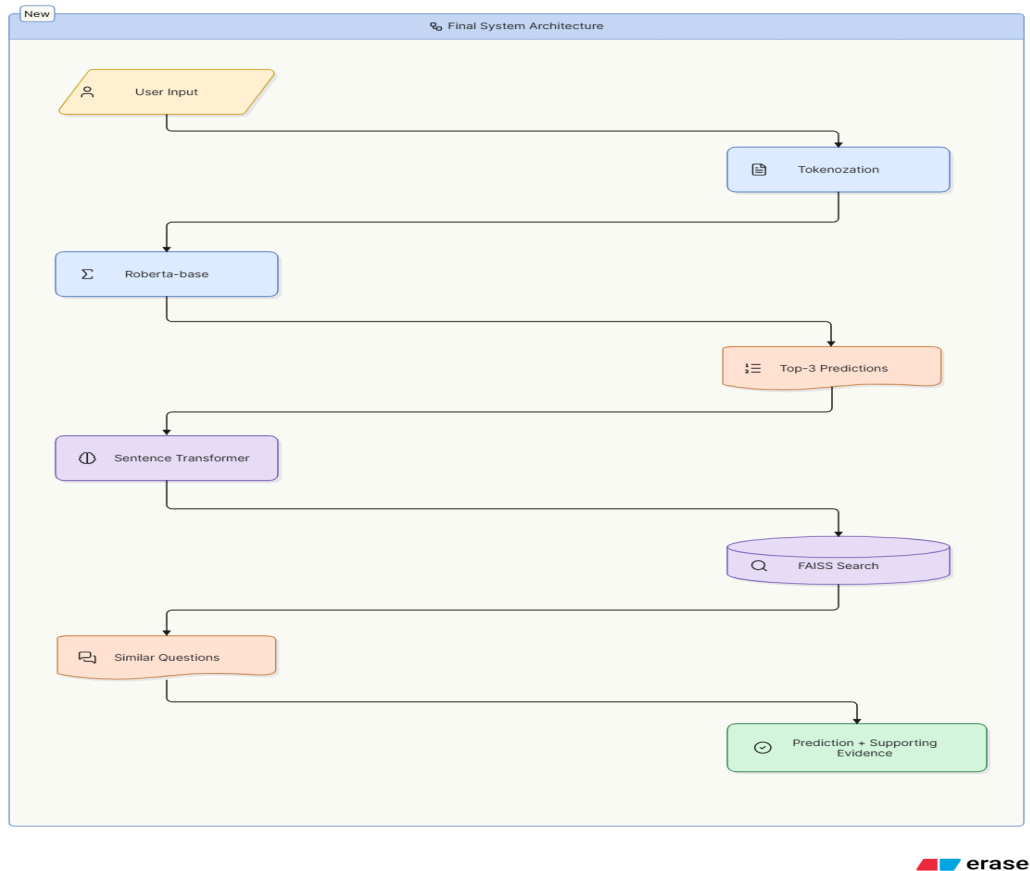
7. Model Development

Model Family	Models Explored	Key Takeaway
Traditional ML	TF-IDF + Logistic Regression, Random Forest, LightGBM, XGBoost	Selected as the first successful model for its strong baseline performance and minimal compute needs; Logistic Regression remained the most efficient lightweight option overall.
Deep Learning	Feed-forward Neural Network, BiLSTM with Attention	Built to understand neural architectures; the attention mechanism improved contextual understanding compared to a simple recurrent network.
Transformers	DistilBERT, DeBERTa, FacebookAI RoBERTa	RoBERTa produced the strongest semantic understanding and best overall ranking performance, but its large model size created deployment challenges on free cloud platforms.

8. Retrieval-Augmented Generation (RAG)

Prediction alone provides limited interpretability. To improve explainability, Retrieval-Augmented Generation was integrated into the system, built on Sentence-Transformer embeddings, a FAISS vector database, and top-k nearest-neighbour retrieval. Instead of generating new text, the retrieval component searches for similar historical MCQs and displays them alongside the prediction results, letting users verify predictions against previously observed examples.

9. Final System Architecture



This architecture provides fast inference while remaining suitable for deployment on resource-constrained environments.

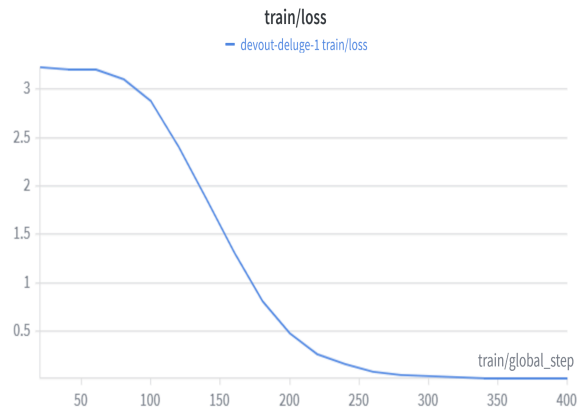
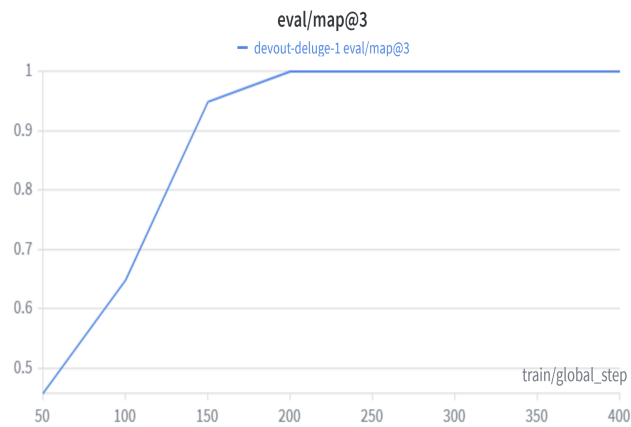
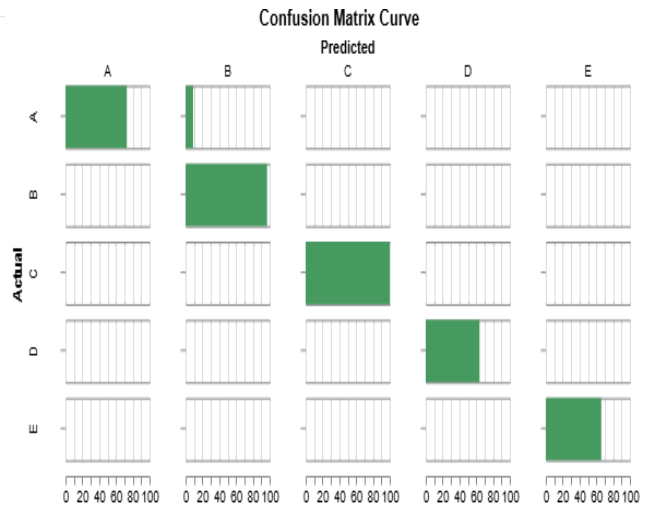
10. Training and Experiment Tracking

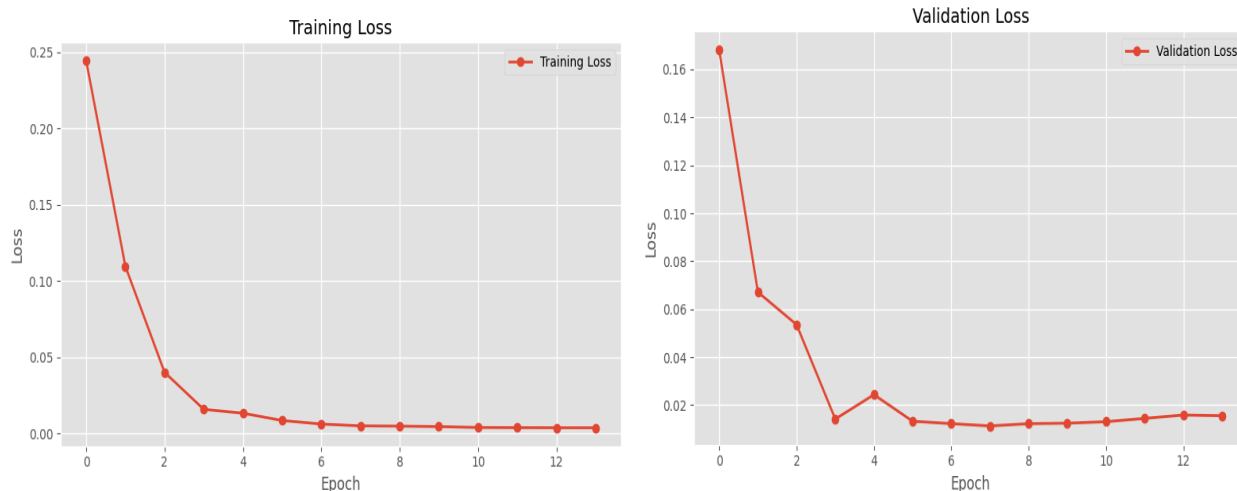
Multiple experiments were conducted throughout the project, evaluating different preprocessing techniques, hyperparameters, and model architectures. Training progress and validation performance were monitored using Weights & Biases (W&B). The following parameters were tuned during experimentation:

- Learning rate
- Batch size
- Number of epochs
- Optimizer
- Sequence length

```
runs.summary["model_comparison"]
```

Filter	
Model	Score
1 TFIDF + Logistic Regression	0.7336
2 SGDClassifier	0.3675
3 RandomForest	0.965
4 LightGBM	0.995
5 XGBoost	0.9925
6 CatBoost	0.98





11. Evaluation

The primary evaluation metric for this project is MAP@3 (Mean Average Precision at 3), which measures how well the correct answer is ranked within the top three predictions. Accuracy, Precision, Recall, and Macro F1 Score were tracked as additional comparison metrics. Overall, Transformer models achieved better semantic understanding, while traditional machine learning offered significantly faster inference.

Model	MAP@3	Accuracy	Relative Inference Speed
TF-IDF + Logistic Regression	0.98452	0.975	Fastest
BiLSTM with Attention	0.993	0.963	Medium
FacebookAI RoBERTa	1.0	1.0	Slowest (largest model)

12. Deployment

A web application was developed using Streamlit, allowing users to enter an MCQ, predict Top-3 answers, view confidence scores, retrieve similar questions, and download prediction results. Deployment initially used the RoBERTa model, but free cloud platforms imposed strict memory limits that made this difficult. The final deployment pipeline was therefore redesigned around TF-IDF + Logistic Regression for prediction and MiniLM + FAISS for retrieval, which reduced memory consumption while maintaining good prediction quality.

13. Challenges Faced

Several technical challenges were encountered during the project, each requiring iterative debugging, experimentation, and optimization experiences that significantly improved practical understanding of ML deployment:

- Large transformer model sizes
- GPU memory limitations
- Hugging Face deployment restrictions
- Free-tier RAM limits
- Package compatibility issues
- Model serialization compatibility
- Streamlit deployment failures

14. Error Analysis

Although the final system performs well, several prediction errors remain. Common failure cases include semantically similar answer choices, ambiguous wording, long question contexts, and highly overlapping distractor options; reviewing these cases helped identify opportunities for future model improvement.

15. Key Learnings

This project provided hands-on experience across the complete machine learning lifecycle, strengthening debugging and problem-solving skills through extensive experimentation. Major learning outcomes include:

- Natural Language Processing
- Feature Engineering
- Machine Learning & Deep Learning
- Transformer Fine-tuning
- Retrieval-Augmented Generation
- FAISS Indexing
- Experiment Tracking
- Cloud Deployment & Model Optimization

16. Future Work

Possible future improvements, aimed at further increasing prediction accuracy while maintaining deployment efficiency, include:

- Larger training datasets
- Quantized transformer models
- ONNX optimization
- LoRA fine-tuning
- Hybrid retrieval methods
- Better reranking algorithms
- Cloud GPU deployment

17. Conclusion

This project successfully demonstrates the complete development lifecycle of an intelligent MCQ solving system. Beginning with exploratory data analysis and traditional machine learning, the work gradually progressed toward Transformer models and Retrieval-Augmented Generation. The final solution balances prediction accuracy, retrieval-based explainability, computational efficiency, and deployment feasibility and, beyond its competitive performance, provides valuable practical experience in modern NLP, deep learning, experiment tracking, and production deployment.

18. References

- Hugging Face Transformers Documentation
- Sentence Transformers Documentation
- FAISS Documentation
- Scikit-learn Documentation
- PyTorch Documentation
- Streamlit Documentation
- Weights & Biases Documentation
- Kaggle Competition Resources

Project Links

- [Live Project](#)
- [Hugging Face Models](#)
- [GitHub Source Code](#)